

Node 调度优化设计方案

GBF Node 调度优化设计方案

版本	修订人	日期	内容
v1.0	王亚雷	2025年12月17日	初版

1. 前期分析

1.1 背景简介

GBF（Generic Business Framework）是一个通用业务流程框架，用于编排和执行业务节点（Node）。当前框架中 Process 的执行采用固定的**顺序循环执行**方式，即按配置顺序依次执行每个 Node。

复制代码

```
// 当前执行方式 - ProcessServiceProvider.java
for (NodeConfig node : config.getNodes()) {
    result = processNodeInvoker.invoke(node, request);
    if (result instanceof ProcessResult) {
        break;
    }
}
```

这种方式简单可靠，但在某些场景下存在性能优化空间：当多个 Node 之间没有依赖关系时，它们本可以并行执行以减少总体耗时。

1.2 目标收益

- 1. 性能提升：通过自动识别无依赖的节点并行执行，减少 Process 整体执行时间
- 2. 零侵入：无需业务方修改任何代码，完全自动化分析依赖关系
- 3. 向前兼容：默认行为与原有逻辑完全一致，通过开关控制新功能
- 4. 双向门：随时可回切到原有执行方式

1.3 需求分析

1.3.1 功能需求

优先级	功能点	描述
P0	依赖自动识别	静态分析 JoinPoint/EndPoint/PredicatePoint 中的 getResponse/getResult 调用
P0	依赖图构建	基于分析结果构建节点依赖图
P0	拓扑排序	将依赖图转换为可执行的层级结构
P0	分层执行	同层无依赖节点并行执行
P0	双向门开关	支持配置开启/关闭分层执行
P1	循环依赖检测	检测并处理循环依赖场景

1.3.2 技术需求

需求项	指标
兼容性	100% 向前兼容，不改变原有行为
启动开销	依赖分析仅在启动时执行一次
运行时开销	分层执行时开销可控
稳定性	分析失败时自动回退顺序执行

1.4 实现分析

1.4.1 前期调研

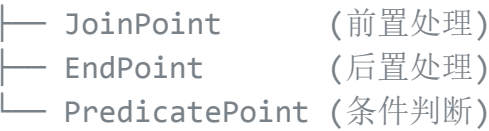
1.4.1.1 依赖来源分析

GBF 框架中节点间依赖的获取**仅通过继承** `ProcessPipeline` 的类实现：

▽

复制代码

ProcessPipeline (提供 `getResponse/getResult` 等方法)



重要发现： `NodeService` 本身不继承 `ProcessPipeline`，因此无需分析 `NodeService` 实现类。

1.4.1.2 技术调研

- **ASM：**成熟的字节码分析框架，可在类加载后分析字节码
- **静态分析：**在启动时一次性分析完成，无运行时开销

1.4.2 差异分析

对比项	原方案	新方案
执行方式	顺序执行	分层并行执行
依赖处理	隐式（顺序保证）	显式（静态分析）
性能	所有节点串行	无依赖节点并行

1.4.3 实现结论

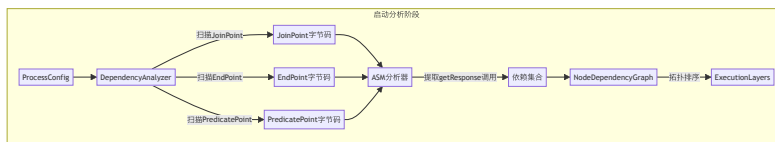
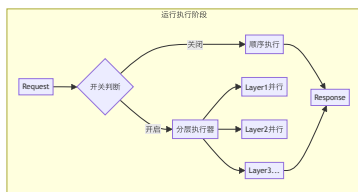
采用纯静态字节码分析方案：

- 启动时使用 ASM 分析 `JoinPoint/EndPoint/PredicatePoint` 的字节码
- 提取 `getResponse` / `getResult` 调用中的节点名称作为依赖
- 构建依赖图并进行拓扑排序，生成执行层级
- 同一层级的节点可并行执行

2. 方案设计

2.1 整体方案

2.1.1 业务架构



2.1.2 系统架构

新增类结构：



复制代码

```
gbf-client/src/main/java/com/amap/mall/gbf/client/
├── analysis/
│   ├── DependencyAnalyzer.java          # 依赖分析入口
│   └── DependencyMethodVisitor.java     # ASM方法访问器
├── graph/
│   ├── NodeDependencyGraph.java        # 依赖图
│   └── ExecutionLayer.java             # 执行层

gbf-core/src/main/java/com/amap/mall/gbf/
├── process/
│   └── LayeredProcessExecutor.java     # 分层执行器
```

修改类：



复制代码

```
gbf-client:
- config/NodeConfig.java          # 增加 analyzedDependencies 字段
- config/ProcessConfig.java       # 增加 executionLayers 字段

gbf-core:
- register/ProcessRegister.java   # 集成依赖分析
- process/ProcessServiceProvider.java # 集成双向门开关
```

2.2 关键方案

2.2.1 数据模型

ExecutionLayer - 执行层

[复制代码](#)

```
@Data
@AllArgsConstructor
public class ExecutionLayer {
    /**
     * 该层包含的节点名称列表
     */
    private List<String> nodeNames;

    /**
     * 是否可并行执行
     */
    private boolean parallelEnabled;
}
```

NodeDependencyGraph - 依赖图

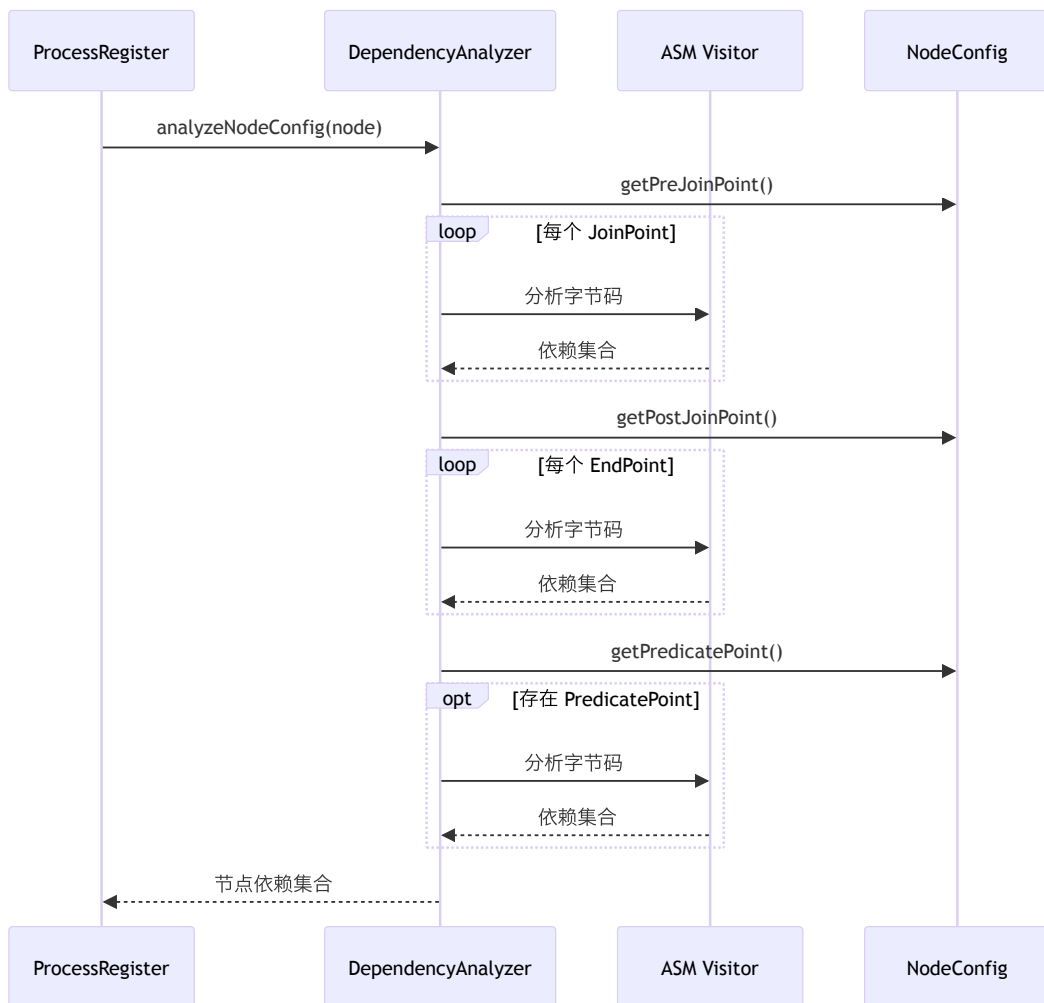
[复制代码](#)

```
public class NodeDependencyGraph {
    // 节点 -> 依赖的前置节点
    private Map<String, Set<String>> dependencies;
    // 原始节点顺序
    private List<String> originalOrder;

    // 拓扑排序生成执行层级
    public List<ExecutionLayer> buildExecutionLayers();
    // 循环依赖检测
    public boolean hasCycle();
}
```

2.2.2 关键逻辑

依赖分析流程



ASM 字节码分析

识别以下方法调用并提取参数：

方法名	参数类型	提取方式
<code>getResponse(String)</code>	String	LDC 指令的字符串常量
<code>getResponse(Class)</code>	Class	LDC 指令的 Type 常量，转换为 beanName
<code>getResult(String)</code>	String	同上
<code>getNonNullResponse(...)</code>	String/Class	同上
<code>getResponseIfPresent(...)</code>	String/Class	同上

拓扑排序算法

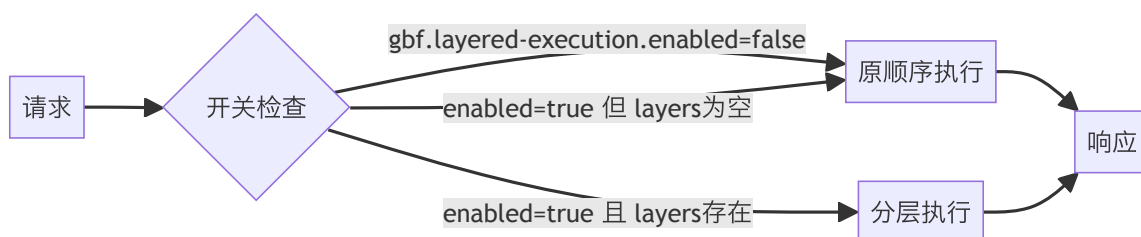
输入：节点列表 + 每个节点的依赖集合

输出：执行层级列表

1. 初始化 `executed = {}`, `remaining = 所有节点`
2. while `remaining` 非空:
 - a. `ready = remaining` 中所有依赖已在 `executed` 中的节点
 - b. if `ready` 为空 且 `remaining` 非空: 存在循环依赖, 回退顺序执行
 - c. `layers.add(new ExecutionLayer(ready, ready.size > 1))`
 - d. `executed.addAll(ready)`
 - e. `remaining.removeAll(ready)`
3. return `layers`

2.2.3 稳定性

双向门设计



异常处理

场景	处理策略
字节码分析失败	该节点视为无依赖，不影响其他节点
循环依赖	打印警告日志，回退顺序执行
依赖节点不存在	过滤掉无效依赖
分层执行异常	向上抛出，与原有行为一致

3. 详细设计

3.1 接口协议

配置项

▼

复制代码

```
gbf:
  layered-execution:
    enabled: false           # 全局开关，默认关闭
    enabled-processes:      # 可选：指定启用的 Process
      - processName1
      - processName2
```

ProcessConfigBuilder 扩展

▼

复制代码

```
// 可选：手动控制单个 Process 的分层执行
public ProcessConfigBuilder layeredExecution(boolean enabled) {
    this.layeredExecution = enabled;
    return this;
}
```

3.2 代码实现

详见「附录 - 完整代码实现」

4. 其他

4.1 测试范围

测试类型	测试内容
单元测试	DependencyMethodVisitor 字节码分析准确性
单元测试	NodeDependencyGraph 拓扑排序正确性

单元测试	循环依赖检测
集成测试	分层执行与顺序执行结果一致性
性能测试	分层执行性能对比

4.2 成本评估

项目	评估
开发工时	3-5 人天
依赖引入	ASM 库（约 300KB）
启动耗时影响	毫秒级（仅分析 Point 类）

4.3 实施计划

阶段	内容	时间
Phase 1	代码实现 + 单元测试	2天
Phase 2	集成测试 + 性能测试	1天
Phase 3	灰度验证（单个 Process）	1天
Phase 4	全量推广	持续

4.4 后续规划

1. 增强分析能力：支持更多依赖获取场景
2. 可视化：提供依赖图可视化工具
3. 智能优化：根据历史执行数据动态调整并行策略

5. 附录

5.1 名词解释

时序对比：



复制代码

原顺序执行： $A(100ms) \rightarrow B(100ms) \rightarrow C(100ms) \rightarrow D(100ms) = 400ms$

分层并行： $[A,B](100ms) \rightarrow [C,D](100ms) = 200ms$

性能提升： 50%

5.3 参考文档

- ASM 官方文档: <https://asm.ow2.io/>
- GBF 框架使用文档